

Python

Основы работы с Python



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Python



Применение:

Написание скриптов для автоматизации задач.

Обработка данных и работа с файлами.

Примеры:

Разработка алгоритмов и математических вычислений.

Создание собственных инструментов для решения специфических задач.

Пример

Ситуация:

Компания разрабатывает инструмент для управления проектами, который должен быть легким, простым в поддержке.

Действия:

Разработчики используют Python для:

- Создания функционала по управлению проектами и задачами с использованием встроенных возможностей языка (работа с файлами, обработка данных, взаимодействие с пользователями через консоль).
- Хранения данных в простых текстовых файлах или формате CSV, управляя ими с помощью стандартных модулей csv и os.
- Создания команд для автоматизации рабочих процессов и генерации отчетов с использованием стандартных функций Python для обработки строк и данных.
- Реализации базовых функций безопасности (например, шифрование паролей) с использованием модуля hashlib.

Результат:

Создано легковесное веб-приложение для управления проектами, которое не требует сложной инфраструктуры и легко поддерживается.

Hard Skills модуля для резюме

Использую Python для работы с данными



Создание автоматизированных скриптов для обработки данных с соблюдением стандарта PEP8.



Библиотеки и технологии: requests (работа с API), pymysql, pymongo, Jupyter Notebook, statistics.



Базы данных: опыт интеграции Python с реляционными (MySQL) и нереляционными (MongoDB) базами данных.

План занятия

- Особенности языка Python
- Интерпретатор и компилятор
- Jupyter Notebook
- Переменная и оператор присваивания
- Правила именования переменных
- Синтаксис, комментарии и PEP 8
- Функции, вызов функции
- Функция print и функция input

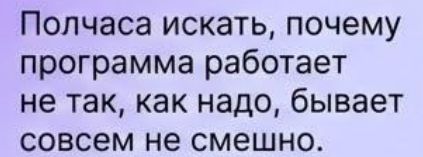


ОСНОВНОЙ БЛОК





Особенности языка Python



Полезно знать



Python

Это высокоуровневый язык программирования, известный своей простотой и читабельностью.



Простота



Динамическая типизация



Интерпретируемый



Кроссплатформенность



Сообщество и экосистема

Python — лучший выбор для аналитика



Готовые решения: множество библиотек для Python позволяют быстро решать задачи аналитика данных — для очистки данных есть Pandas, для графиков — Seaborn, для нейросетей — PyTorch.

Живое сообщество: почти на любой вопрос вы найдете ответ на Stack Overflow.

Интерпретатор и компилятор



Компилятор

Программа, которая сначала переводит весь исходный код в машинный код, а затем запускает его.

Компилятор преобразует весь код сразу.

Интерпретатор

Программа, которая выполняет код построчно, сразу переводя его в машинный код.

Интерпретатор исполняет код по мере чтения.

Python — интерпретируемый язык



Позволяет запускать программы без предварительной компиляции.
Это ускоряет разработку и упрощает отладку, но может быть медленнее по сравнению с компилируемыми языками.

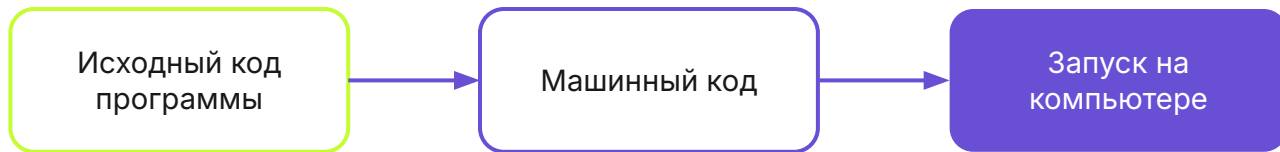


Машинный код

Это низкоуровневые инструкции, которые процессор компьютера может напрямую выполнять.

Он состоит из бинарных данных (0 и 1) и специфичен для каждого типа процессора.

Машинный код





ВОПРОСЫ





Jupyter Notebook



Jupyter Notebook

Это интерактивная среда разработки и обмена документами, которая позволяет создавать и выполнять код Python в виде ячеек.

Jupyter Notebook



Состоит из веб-интерфейса, который позволяет создавать и редактировать ноутбуки, а также встроенного интерпретатора Python, который выполняет код в каждой ячейке ноутбука. Каждая ячейка может содержать код, текстовые описания, формулы, изображения, графики и другие элементы.

Jupyter Notebook



Преимущества

Разработчик может выполнять код в ячейках по одной или несколько сразу, что позволяет получать немедленные результаты и видеть вывод прямо в ноутбук.



Интерактивность



Удобно для тестирования
и экспериментирования с кодом



ВОПРОСЫ





Переменная

Экспресс-опрос



Как вы думаете, что такое переменная?



Что такое данные?

Полезно знать



Переменная

- Это именованная область памяти, в которой хранятся данные.
- После создания переменной можно получить доступ к хранящимся в ней данным по заданному имени.

Данные

- Это информация, которая может быть обработана и сохранена компьютером.
- В программировании данные представляют собой любые значения, которые программа использует для выполнения операций с ними.

Пример



Целые числа: **5, -3, 42**



Числа с плавающей запятой или вещественные: **3.14, -0.001, 2.0**



Строки: **"Привет", 'Мир'**



Данные

Данные могут поступать из разных источников, таких как файлы, базы данных, пользовательский ввод или внешние устройства.

Программы обрабатывают эти данные, изменяют их, анализируют или сохраняют для дальнейшего использования.



ВОПРОСЫ





Оператор 
присваивания

operator и statement



В английском языке:

- Оператор (operator) — возвращает значение
- Инструкция (statement) — выполняет некоторое действие в программе

В русскоязычной литературе часто эти понятия смешиваются, и далее мы будем везде использовать название оператор.

Оператор присваивания



Определение

- Это символ, который используется для присвоения значения переменной.
- В Python оператор присваивания обозначается знаком `=`. Он связывает переменную с заданным значением.

Пример

```
x = 10 # Присваиваем переменной x число 10
name = "John" # Присваиваем переменной name строку "John"
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите правильный вариант ответа

Оператор присваивания в Python обозначается символом:

- a. ==
- b. =
- c. #



Выберите правильный вариант ответа

Оператор присваивания в Python обозначается символом:

- a. ==
- b. =**
- c. #

Соотнесите термин с его определением:

- | | |
|--------------------------|--|
| 1. Компилятор | a. Это программа, которая выполняет код построчно. |
| 2. Интерпретатор | b. Это низкоуровневый код, который исполняется компьютером напрямую. |
| 3. Оператор присваивания | c. Это программа, которая переводит весь код в машинный до его выполнения. |
| 4. Машинный код | d. Это символ, связывающий переменную с определённым значением. |

Соотнесите термин с его определением:

- | | |
|--------------------------|--|
| 1. Компилятор | с. Это программа, которая переводит весь код в машинный до его выполнения. |
| 2. Интерпретатор | а. Это программа, которая выполняет код построчно. |
| 3. Оператор присваивания | д. Это низкоуровневый код, который исполняется компьютером напрямую. |
| 4. Машинный код | б. Это символ, связывающий переменную с определённым значением. |



ВОПРОСЫ





Правила именован переменных

Правила для чистого и понятного кода:

-  Имя переменной может содержать только буквы, цифры и знак подчеркивания _
-  Начинается только с буквы или знака подчеркивания. (1number не подходит)
-  Переменные регистрозависимы ("age" и "Age" - разные переменные)
-  Не используйте зарезервированные слова (как if, else, while и т.д.)
-  Не используйте названия функций (как print, int, max и т.д.)

Правила для чистого и понятного кода:



В Python переменные принято записывать в формате snake_case



По имени переменной должно быть понятно что в ней хранится



Имя переменной должно быть достаточно длинным, чтобы передавать смысл



Но не слишком длинным. Это улучшает читаемость

Правила именования переменных



Правильные имена переменных:

`coordinate1 = 42`

`user_name = "Alice"`

`_price = 10`

Неправильные имена переменных:

`2nd_coordinate = 33` # Начинается с цифры

`my-variable = 10` # Содержит дефис

`my variable = 10` # Содержит пробел

`total%items = 5` # Содержит недопустимый символ "%"

`if = 42` # Использует зарезервированное слово "if"

Переменные в Python — как коробки



Вы можете положить туда число, текст или целый список покупок.

Главное — не называть коробку a, b или c.

Потому что через неделю вы будете смотреть на свой код и думать:
«А что лежало в коробке "a"? Смысл жизни или цена моркови?»

«А что лежало в коробке "a"?
Смысл жизни или цена моркови?»





ВОПРОСЫ





ЗАДАНИЕ





Выберете вариант ответа

Определите, какие из имен переменных являются корректными:

- a. student_name
- b. age_25
- c. 2nd_place
- d. is_student
- e. my-variable
- f. if
- g. user_@ge
- h. total_score
- i. num 1
- j. name!



Выберете вариант ответа

Определите, какие из имен переменных являются корректными:

- a. **student_name**
- b. **age_25**
- c. 2nd_place
- d. **is_student**
- e. my-variable
- f. if
- g. user_@ge
- h. **total_score**
- i. num 1
- j. name!



ВОПРОСЫ





Синтаксис





Синтаксис

Это набор правил, определяющих, как должны быть организованы конструкции в языке программирования.

Синтаксис в Python включает:



Структуру кода



Идентификаторы



Строки и комментарии



Операторы

Некоторые правила синтаксиса



Конец строки

Конец строки является концом инструкции.

Пример

```
num1 = 5
num2 = 7
```

Некоторые правила синтаксиса



В одной строке

Иногда возможно записать несколько инструкций в одной строке, разделяя их точкой с запятой.

Однако это не рекомендуется.

Пример

```
num1 = 5; num2 = 7
```



Ошибки синтаксиса

(`SyntaxError`) — это ошибки, возникающие когда код не соответствует правилам языка. Эти ошибки препятствуют интерпретатору понять и выполнить код.

Ошибки синтаксиса



При запуске программы с ошибкой синтаксиса интерпретатор Python остановит выполнение кода и выведет сообщение об ошибке.

Сообщение об ошибке включает:



Тип ошибки



Описание ошибки



Номер строки



Строка кода

Рассмотрим ошибку



Код

```
1number = 5
```

Решение

При попытке запустить программу вы получите сообщение об ошибке

Недопустимые символы

```
1number = 5
```

```
Cell In[9], line 1
```

```
1number = 5
```

```
^
```

```
SyntaxError: invalid decimal literal
```

Неправильное использование кавычек для строк

```
name = "John'
```

```
Cell In[10], line 1
```

```
    name = "John'
```

```
        ^
```

```
SyntaxError: unterminated string literal (detected at line 1)
```

Неправильное использование операторов

```
1+*2
```

```
Cell In[2], line 1
```

```
1+*2
```

```
^
```

```
SyntaxError: invalid syntax
```

Ошибки — это часть обучения и работы



Даже сеньор-разработчики с 10-летним стажем регулярно видят `SyntaxError`.

Python — ваш **помощник**: он сам подсказывает, где именно вы ошиблись, чтобы было проще улучшить код.





ВОПРОСЫ





Комментарии



Комментарии

используются в Python для пояснения кода и делают его более понятным. Они не влияют на выполнение программы, так как интерпретатор игнорирует их.

Основные правила



В одной строке

- Начинаются с символа # и продолжаются до конца строки.
- Используются для кратких пояснений.

Пример

Это комментарий

x = 5 # Присваиваем значение 5 переменной x

Комментарии — это письма из прошлого



Код говорит компьютеру,
как работать



Комментарии говорят человеку,
зачем это нужно



ВОПРОСЫ





PEP 8





PEP 8 (Python Enhancement Proposal 8)

Это документ, который содержит рекомендации по стилю написания кода на Python. Его цель — улучшить читаемость кода.

PEP 8



Соблюдение PEP 8 помогает разработчикам писать чистый и понятный код, который легче поддерживать и развивать.

Основные рекомендации PEP 8:



Длина строки



Пробелы



Именованье



Комментарии



ВОПРОСЫ





ЗАДАНИЕ





**Какой результат выведет
следующий код?**

$a = 3$

$b = 2a + 1$

- a. 3 7
- b. Ошибка
- c. 7
- d. $2a + 1$



Какой результат выведет
следующий код?

$a = 3$

$b = 2a + 1$

a. 3 7

b. Ошибка

c. 7

d. $2a + 1$



Выберите вариант ответа:

Для создания комментария в Python можно использовать:

- a. `/*`
- b. `#`
- c. `'''`
- d. `//`



Выберите вариант ответа:

Для создания комментария в Python можно использовать:

- a. /*
- b. #**
- c. '''
- d. //



ВОПРОСЫ





Функции



Функция

Это заранее написанный блок кода, который выполняет определённую задачу. В Python есть встроенные функции, которые можно использовать, просто вызвав их по имени.



Вызов функции

Это процесс выполнения функции в коде.

Чтобы вызвать функцию, нужно:



Указать её имя



Указать круглые скобки сразу после имени (они инициируют выполнение функции)



Передать необходимые данные (аргументы) в круглых скобках, при необходимости

Вызов функции



Код

```
print("Привет, мир!")
```

Разбор

print — это имя функции.

() — вызов функции.

"Привет, мир!" — это аргумент, который передаётся функции.



ВОПРОСЫ





ЗАДАНИЕ





Что из перечисленного делает вызов функции?

- a. создает новую функцию.
- b. выполняет ранее написанную функцию.
- c. определяет переменную.
- d. завершает программу.



Что из перечисленного делает вызов функции?

- a. создает новую функцию.
- b. выполняет ранее написанную функцию.**
- c. определяет переменную.
- d. завершает программу.



ВОПРОСЫ





Функция print



Функция print

Это встроенная функция в Python, которая выводит информацию на экран (в консоль). Она может выводить текст, числа, переменные и другие типы данных.

Пример: `print("Привет, мир!")`

```
print("Привет, мир!")
```

Привет, мир!

Пример: `print("Число:", 10)`

```
print("Число:", 10)
```

Число: 10



ВОПРОСЫ





Функция input



Функция input

Это встроенная функция в Python, которая позволяет получать данные от пользователя через ввод с клавиатуры.

Функция input



Когда вызывается `input()`, программа приостанавливается и ждёт, пока пользователь введёт текст и нажмёт Enter. На месте вызова функции появляется возвращаемое значение, которое далее можно присвоить переменной или передать в другую функцию.

Пример использования



Код

```
name = input("Введите ваше имя: ")  
print("Привет,", name)
```

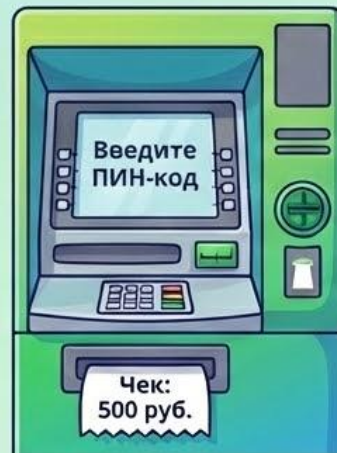
Разбор

- Функция input выводит строку "Введите ваше имя: " и ждет ввода от пользователя.
- Введённый текст сохраняется в переменной name.
- С помощью print выводится приветствие с именем пользователя.

Вопрос

Как вы думаете, а вы сами сегодня уже встречали результат работы функций, аналогичных print и input?

Где в вашей жизни "спрятаны" эти две команды?





ВОПРОСЫ

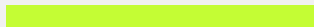




ЗАДАНИЕ

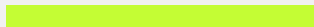


Соотнесите функцию с её описанием:



- | | |
|-------------------------|---|
| 1. <code>print()</code> | a. Получает данные от пользователя через ввод с клавиатуры. |
| 2. <code>input()</code> | b. Выводит информацию на экран. |

Соотнесите функцию с её описанием:

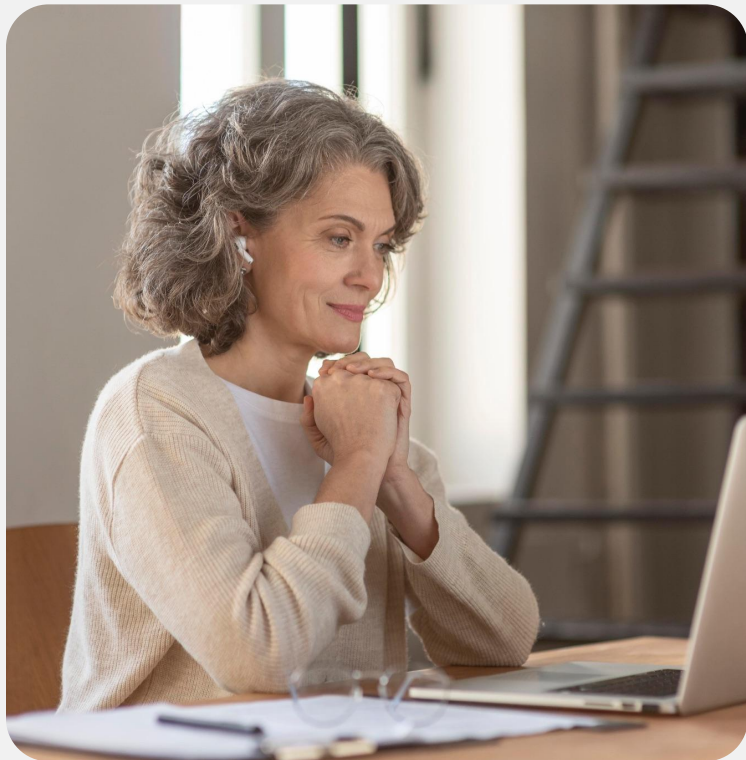


- | | |
|-------------------------|---|
| 1. <code>print()</code> | b. Выводит информацию на экран. |
| 2. <code>input()</code> | a. Получает данные от пользователя через ввод с клавиатуры. |



Напишите программу:

Программа должна вывести на экран строку:
"Hello, Python!".



Напишите программу:

Напишите программу, которая попросит пользователя ввести его возраст, а затем выведет его на экран.



ВОПРОСЫ



Домашнее задание

1. Приветственное сообщение

Создайте программу, которая выведет строки "Hi!" и "Hello!" на разных строках.

2. Именное приветствие

Создайте программу, которая попросит пользователя ввести его имя, а затем выведет фразу: "Hello, <Name>" на новой строке.

Заключение

Сегодня мы



- Познакомились с Jupyter Notebook: скоро он станет полем для экспериментов с данными.



- Освоили переменные: вы поняли, как сохранять информацию в памяти компьютера — это первый шаг к автоматизации огромных отчетов.



- Разобрались в типах данных.



- Узнали, что такое функции и научились их использовать.

К концу модуля вы будете



Писать чистый
и профессиональный
код



Работать с базами
данных напрямую из
Python



Автоматизировать
сбор данных через API



Создавать сложные
логические системы

Помните: каждый крутой дата-аналитик из Google или Amazon когда-то сидел и так же, как вы сегодня, радовался своей первой строчке `print("Hello World")`. Вы на правильном пути!

Заключение

